

1. Adding full-text search to the blog

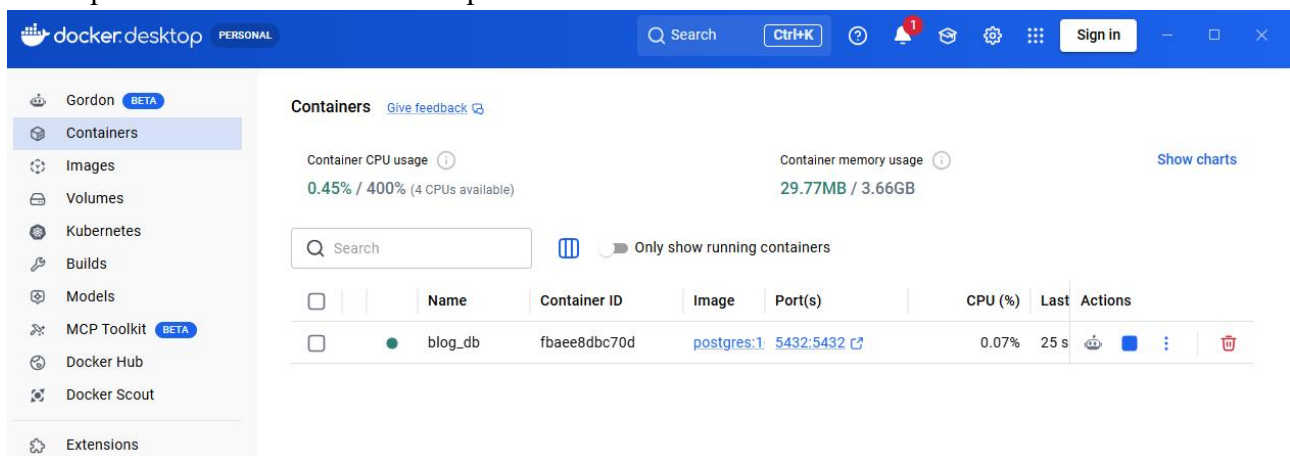
- Django ORM supports simple search using filters like `contains` and `icontains`.
- Example: `Post.objects.filter(body__contains='framework')` searches text in fields.
- For advanced search (relevance, similarity, weighted fields), a **full-text search engine** is needed.
- Full-text search analyzes **actual words**, not just string matching.
- Django supports PostgreSQL full-text search via `django.contrib.postgres`.
- This module exposes advanced PostgreSQL features not available in other databases.
- SQLite has **limited full-text search support** and isn't fully supported by Django for this.
- PostgreSQL is **better suited for production and advanced search features**.
- Data can be migrated from SQLite to PostgreSQL to enable full-text search.
- For production, use robust databases like **PostgreSQL, MySQL, MariaDB, or Oracle**.
- PostgreSQL can be easily deployed using a **Docker image**.

a. Installing Docker

- **Docker** is an **open-source containerization platform**.
- It allows developers to **package applications into containers**.
- Containers simplify **building, running, managing, and distributing apps**.
- Need to **download and install Docker** for your operating system (Linux, macOS, Windows):
<https://docs.docker.com/get-docker/>

b. Installing PostgreSQL

- Open Visual Studio Code
- Go to Terminal → New Terminal
- Run: `docker pull postgres:16.2`
- Execute the following command in the shell to start the PostgreSQL Docker container [**Docker should be started**, so the command can run.]:
`docker run --name=blog_db -e POSTGRES_DB=blog -e POSTGRES_USER=bloguser -e POSTGRES_PASSWORD="Abcd1234#" -p 5432:5432 -d postgres:16.2`
- Replaced `xxxxxx` with the actual password `Abcd1234#`.



- **Check its status** - Run: `docker ps -a`
- A new Docker container named `blog_db` is running.
- We can **stop, restart, or delete** the container from the Actions menu.
- **Warning:** Deleting the container will also delete the database and all stored data.
- Install the PostgreSQL Python adapter using: `python -m pip install psycopg==3.1.18`
- Install it right there in your active environment
- The next step is to **migrate existing data from SQLite to PostgreSQL**.

c. Dumping the existing data

- Before switching databases, **export existing SQLite data**.
- Plan: ***Dump data → switch to PostgreSQL → import data.***
- Django uses **fixtures** to handle data export/import.
 - Supported formats: JSON, XML, YAML
 - Here, JSON is used
- The dumpdata command:
 - Exports database data (default: JSON)
 - Includes model and field information
 - Outputs to standard output unless a file is specified
- Useful options:
 - Limit to specific apps or models (app.Model)
 - --format → choose format
 - --output → save to file
 - --indent → format JSON for readability
- Command to export all data:

python manage.py dumpdata --indent=2 --output=mysite_data.json

- Result:
 - Data saved in ***mysite_data.json***
 - Contains all models and their records
- ```
[
 {
 "model": "admin.logentry",
 "pk": 1,
 "fields": {
 "action_time": "2026-03-06T12:54:48.701Z",
 "user": 1,
 "content_type": 7,
 "object_id": "1",
 "object_repr": "Hello Students",
 "action_flag": 1,
 "change_message": "[{\"added\": {}}]"
 }
 },
 ...
```

- If encoding errors occur, use UTF-8 mode:

***python -Xutf8 manage.py dumpdata --indent=2 --output=mysite\_data.json***

- Next step:
  - Switch Django database to PostgreSQL
  - Import the exported data into the new database

#### d. Switching the database in the project

- Edit the *settings.py* file of your project and modify the **DATABASES** setting to make it look as follows.

**File: Open mysite/settings.py**

```
DATABASES = {
 'default': {
 'ENGINE': 'django.db.backends.postgresql',
 'NAME': config('DB_NAME'),
 'USER': config('DB_USER'),
 'PASSWORD': config('DB_PASSWORD'),
 'HOST': config('DB_HOST'),
 'PORT': '5432',
 }
}
```

- Database engine switched to **PostgreSQL**
- Database credentials are now managed using **environment variables** via python-decouple
- Update the *.env file* with:

```
EMAIL_HOST_USER=your_account@gmail.com
EMAIL_HOST_PASSWORD=xxxxxxxxxxxxx
DEFAULT_FROM_EMAIL=My Blog <your_account@gmail.com>
DB_NAME=blog
DB_USER=bloguser
DB_PASSWORD=Abcd1234#
DB_HOST=localhost
```

- Replace placeholder passwords with your actual PostgreSQL container password
- The new PostgreSQL database starts **empty**
- Run migrations to set up database schema: ***python manage.py migrate***

Operations to perform:

Apply all migrations: admin, auth, blog, contenttypes, sessions, sites, taggit

Running migrations:

Applying contenttypes.0001\_initial... OK

Applying auth.0001\_initial... OK

...

- If error exists **Step 1: Remove old version: *pip uninstall psycopg***  
**Step 2: Install latest compatible binary version: *pip install "psycopg[binary]"***  
**And *python -m pip install "psycopg[binary]==3.1.18"***
- If still error exists **Step 1: Restart environment: *deactivate* and *i***  
**Step 2: Run Django: *python manage.py migrate***
- Migration results:
  - Applies all migrations for apps like admin, auth, blog, sessions, etc.
  - Each migration shows OK when successfully applied
- After migrations:
  - PostgreSQL database structure matches Django models
  - Project is now connected to PostgreSQL
- Next step:
  - **Import previously exported data** from SQLite into PostgreSQL

#### e. Loading the data into the new database

- Run the following command to load the previously exported data into the PostgreSQL database:  
***python manage.py loaddata mysite\_data.json***
- Output: ***Installed 56 object(s) from 1 fixture(s)***
- The number of objects might differ, depending on the users, posts, comments, and other objects that have been created in the database.
- Run: ***python manage.py runserver***
- Open: ***http://127.0.0.1:8000/admin/blog/post/*** in the browser to verify that all posts have been loaded into the new database.

Select post to change

|                                                                                                 |                                        |                        |        |                           |     |            |
|-------------------------------------------------------------------------------------------------|----------------------------------------|------------------------|--------|---------------------------|-----|------------|
| Q <input type="text"/> <input type="button" value="Search"/>                                    |                                        |                        |        |                           |     |            |
| ◀ All dates February 2026 March 2026                                                            |                                        |                        |        |                           |     |            |
| Action: <input type="button" value="-----"/> <input type="button" value="Run"/> 0 of 7 selected |                                        |                        |        |                           |     |            |
| <input type="checkbox"/>                                                                        | TITLE                                  | SLUG                   | AUTHOR | PUBLISH                   | 2 ▲ | STATUS 1 ▲ |
| <input type="checkbox"/>                                                                        | <a href="#">What is Django?</a>        | what-is-django         | Admin  | Feb. 26, 2026, 6:51 a.m.  |     | Draft      |
| <input type="checkbox"/>                                                                        | <a href="#">asas asadas</a>            | asas-asadas            | Admin  | Feb. 27, 2026, 6:11 a.m.  |     | Draft      |
| <input type="checkbox"/>                                                                        | <a href="#">Sample Post</a>            | add                    | Admin  | Feb. 26, 2026, 5:38 a.m.  |     | Published  |
| <input type="checkbox"/>                                                                        | <a href="#">First Publish</a>          | first-publish          | Admin  | March 6, 2026, 7:49 a.m.  |     | Published  |
| <input type="checkbox"/>                                                                        | <a href="#">Post 2</a>                 | post-2                 | Admin  | March 6, 2026, 7:56 a.m.  |     | Published  |
| <input type="checkbox"/>                                                                        | <a href="#">This is my sample Post</a> | this-is-my-sample-post | Admin  | March 20, 2026, 5:33 a.m. |     | Published  |
| <input type="checkbox"/>                                                                        | <a href="#">Markdown Post</a>          | markdown-post          | Admin  | March 28, 2026, 6:32 a.m. |     | Published  |

#### f. Simple search lookups

- Edit the ***settings.py*** file of your project and add ***django.contrib.postgres*** to the **INSTALLED\_APPS** setting.

**File: Open mysite/settings.py**

```
INSTALLED_APPS = [
 ...
 'django.contrib.staticfiles',
 'django.contrib.postgres',
 'taggit',
 ...
]
```

- **Run: *python manage.py shell***
- We can search against a single field using the search QuerySet lookup.

```
>>> from blog.models import Post
>>> Post.objects.filter(title__search='django')
<QuerySet []>
>>> Post.objects.filter(title__search='Hello')
<QuerySet [<Post: Hello Guys>, <Post: Hello Students>]>
```

#### Searching against multiple fields

- We have to define a SearchVector object for the title and body fields of the Post model.  

```
>>> from django.contrib.postgres.search import SearchVector
>>> from blog.models import Post
```

```
>>> Post.objects.annotate(search=SearchVector('title', 'body'),).filter(search='django')
<QuerySet []>
>>> Post.objects.annotate(search=SearchVector('title', 'body'),).filter(search='Hello')
<QuerySet [<Post: Hello Guys>, <Post: Hello Students>]>
```

#### g. Building a search view

- Create a custom view to allow your users to search posts.

**File: Open blog/forms.py**

```
class SearchForm(forms.Form):
 query = forms.CharField()
```

**File: Open blog/views.py**

```
...
from django.contrib.postgres.search import SearchVector
from .forms import CommentForm, EmailPostForm, SearchForm
...
def post_search(request):
 form = SearchForm()
 query = None
 results = []
 if 'query' in request.GET:
 form = SearchForm(request.GET)
 if form.is_valid():
 query = form.cleaned_data['query']
 results = (
 Post.published.annotate(
 search=SearchVector('title', 'body'),
)
 .filter(search=query)
)
 return render(
 request,
 'blog/post/search.html',
 {
 'form': form,
 'query': query,
 'results': results
 }
)
```

- Uses **GET request** → query appears in URL
- Validates form input
- Searches in: title, body
- Uses PostgreSQL full-text search (SearchVector)
- Create a new file inside the *templates/blog/post/* directory, name it *search.html* and add the following code to it –

```
{% extends "blog/base.html" %}
{% load blog_tags %}
{% block title %}Search{% endblock %}
{% block content %}
```

```

{% if query %}
<h1>Posts containing "{{ query }}"</h1>
<h3>
 {% with results.count as total_results %}
 Found {{ total_results }} result{{ total_results|pluralize }}
 {% endwith %}
</h3>
{% for post in results %}
 <h4>

 {{ post.title }}

 </h4>
 {{ post.body|markdown|truncatewords_html:12 }}
{% empty %}
<p>There are no results for your query.</p>
{% endfor %}
<p>Search again</p>
{% else %}
<h1>Search for posts</h1>
<form method="get">
 {{ form.as_p }}
 <input type="submit" value="Search">
</form>
{% endif %}
{% endblock %}

```

- Add URL Route (urls.py)

**File: Open blog/urls.py**

```

urlpatterns = [
 ...
 path('feed/', LatestPostsFeed(), name='post_feed'),
 path('search/', views.post_search, name='post_search'),
]

```

- Run: ***python manage.py runserver***
- Open: ***http://127.0.0.1:8000/blog/search/*** in the browser

## Search for posts

Query:

- Enter a Query “Hello” and Click on Search Button.

## Posts containing "Hello"

Found 2 results

[Hello Guys](#)

Hello Guys, How Are You?

[Hello Students](#)

Hello, Hi How Are You?

[Search again](#)

### h. Stemming and ranking results

- **Stemming** reduces words to a **base/root form**
- Example: *music*, *musical*, *musicality* → **music**
- Helps match similar words automatically
- **Stop Words Removal**
- Common words are ignored: *a*, *the*, *on*, *of*
- Improves search relevance by removing noise

**File: Open blog/views.py**

```
from django.contrib.postgres.search import (
 SearchVector,
 SearchQuery,
 SearchRank
)
...
def post_search(request):
 ...
 if form.is_valid():
 query = form.cleaned_data['query']
 search_vector = SearchVector('title', 'body')
 search_query = SearchQuery(query)
 results = (
 Post.published.annotate(
 search=search_vector,
 rank=SearchRank(search_vector, search_query)
)
 .filter(search=search_query)
 .order_by('-rank')
)
 return render(
 ...
)
```

- Run: ***python manage.py runserver***
- Open: ***http://127.0.0.1:8000/blog/search/*** in the browser and see the difference (Results are Ranked).

## Posts containing "Hello"

Found 2 results

[Hello Students](#)

Hello, Hi How Are You?

[Hello Guys](#)

Hello Guys, How Are You?

[Search again](#)

### i. Stemming and removing stop words in different languages

- We can change the language used for: Stemming, Stop-word removal
- Done using the config parameter in: SearchVector and SearchQuery

#### Example (Spanish Search) in *views.py*

```
search_vector = SearchVector('title', 'body', config='spanish')
search_query = SearchQuery(query, config='spanish')
results = (
 Post.published.annotate(
 search=search_vector,
 rank=SearchRank(search_vector, search_query)
)
 .filter(search=search_query)
 .order_by('-rank')
)
```

### j. Weighting queries

- We can boost specific vectors so that more weight is attributed to them when ordering results by relevancy.

#### File: *Open blog/views.py*

```
def post_search(request):
 ...
 search_vector = SearchVector(
 'title', weight='A'
) + SearchVector('body', weight='B')
 search_query = SearchQuery(query)
 results = (
 ...
)
 .filter(rank__gte=0.3)
 .order_by('-rank')
)
return render(
 ...
)
```



- A **search vector** is created combining:
  - title with weight **A (1.0)** → highest importance
  - body with weight **B (0.4)** → lower importance
- This ensures matches in the **title** are considered more relevant than matches in the **body**
- Only results with **rank**  $\geq 0.3$  are included
- This removes weak or less relevant matches

#### k. Searching with trigram similarity

- A **trigram** is a group of **three consecutive characters**.
- We can measure the similarity of two strings by counting the number of trigrams that they share.
- This approach turns out to be very effective for measuring the similarity of words in many languages.
- First, execute the following command in the shell prompt to create an empty migration:
 

```
python manage.py makemigrations --name=trigram_ext --empty blog
```
- This will create an empty migration for the blog application.

Migrations for 'blog':

blog\migrations\0005\_trigram\_ext.py

**File: Open blog/migrations/0005\_trigram\_ext.py**

```
from django.contrib.postgres.operations import TrigramExtension
from django.db import migrations
class Migration(migrations.Migration):
 dependencies = [
 ('blog', '0004_post_tags'),
]
 operations = [
 TrigramExtension()
]
```

- We have added the *TrigramExtension* operation to the database migration, which executes the SQL statement *CREATE EXTENSION pg\_trgm* to create the extension in PostgreSQL.
- Run: *python manage.py migrate blog*

Operations to perform:

Apply all migrations: blog

Running migrations:

Applying blog.0005\_trigram\_ext... OK

**File: Open blog/views.py**

```
from django.contrib.postgres.search import TrigramSimilarity
...
def post_search(request):
 ...
 results = (
 Post.published.annotate(
 similarity=TrigramSimilarity('title', query),
)
 .filter(similarity__gt=0.1)
 .order_by('-similarity')
)
```

return render(...

- Run: ***python manage.py runserver***
- Open: ***http://127.0.0.1:8000/blog/search/*** in the browser and search for “Yello”, “Stu”, “Guy”, etc.

### Posts containing "Yello"

Found 2 results

[Hello Guys](#)

Hello Guys, How Are You?

[Hello Students](#)

Hello, Hi How Are You?

[Search again](#)

### Posts containing "Stu"

Found 1 result

[Hello Students](#)

Hello, Hi How Are You?

[Search again](#)

**My blog**

### Posts containing "guy"

Found 1 result

[Hello Guys](#)

Hello Guys, How Are You?

[Search again](#)

**My blog**